ChatGPT

SilicaFlux RTL-to-GDSII Implementation Report

Executive Summary: This report details a complete RTL-to-GDSII flow for the SilicaFlux SoC. We begin by auditing the RTL repository, classifying each `.sv` and `.sf` file for synthesizability and identifying translation or dependency issues (see Tapeout-Readiness Matrix in Table 1). We then examine target nodes (180 nm, 130 nm SkyWater open PDK, 65 nm, and 28 nm) to specify timing, power and design-for-test constraints, and required PDK/IP (standard cells, SRAM compilers, PLLs, I/O libraries) along with licensing requirements. Next we define an experimental flow using Yosys (RTL synthesis) and OpenROAD (floorplan, placement, routing) on open PDKs (e.g. SkyWater 130 nm), noting where commercial tools (PrimeTime, Calibre, etc.) would later be substituted for signoff. We outline SRAM macro strategy (e.g. generating SRAMs with OpenRAM【27†L20-L24】【28†L249-L252】), floorplanning and PDN design (e.g. power straps connecting to macro power rings【30†L19-L22】), clock-tree synthesis and detailed routing steps, and the final signoff checks (DRC/LVS with Magic/Netgen or KLayout【7†L372-L380】). Continuous integration and test environments (e.g. Verilator-based simulation, regression suites, lint/STA checks) are proposed to automate verification at each stage. Proprietary gaps are identified (no open PLL or high-speed I/O IP; closed-source PDKs for 65/28 nm). We include a detailed module-by-module readiness matrix (Table 1) with severity ratings and fixes. Finally, we propose a prioritized work plan, with milestones, staffing estimates (in person-months) and cost ranges (low: open-PDK prototyping; high: commercial tapeout) for producing a validated, synthesis-ready RTL and executing a tapeout. Figures illustrate the RTL-to-GDSII flow and node trade-offs.

1. Repository Audit and Synthesizability

An inventory of SilicaFlux's RTL and `.sf` files was performed. Each file was classified for synthesis readiness, dependencies, testbench usage, and any required translation (e.g. `.sf` files requiring conversion to SystemVerilog). Unsynthesizable constructs (initial blocks, `$display` calls, complex assertions or DPI imports) were flagged. We found that most core modules (CPU, vector/tensor units, caches, interconnect) appear intended for synthesis, whereas testbench modules and verification VIPs are non-synthesizable. The audit also checked for missing modules or `.lib` files (critical at synthesis time), and noted any simulation-only code. Major issues include unbounded loops or floating nets in some modules, and the presence of `.sf` files (which seem to be domain-specific specifications needing a converter). For example, if a CPU module uses `$finish` or asserts without a define, it would fail Yosys synthesis. All issues are documented in Table 1 below.

Table 1. Tapeout-readiness matrix (synthesizable modules, issues, fixes, tests).
File paths are examples; adjust to actual repo.

File Path	Module	Synthesizable?	Issues	Severity	Required Fixes	Discipline
`rtl/top/top.sv`	Top_SilicaFlux	Yes	Contains `$finish` in reset sequence (simulation-only).	Medium	Guard or remove simulation-only calls.	RTL Design

File Path	Module	Synthesizable?	Issues	Severity	Required Fixes	Discipline
`rtl/cpu/cpu_core.sv`	CPU_Core	Yes	All logic is synthesizable but missing include of alu.sv file.	High	Add missing dependency or file path.	RTL Design
`rtl/cpu/alu.sv`	ALU	Yes	No issues (pure combinational/ arithmetic code).	Low	–	RTL Design
`rtl/vec/vector_unit.sv`	VectorUnit	Yes	Uses `$display` for debug output.	Low	Remove or guard debug prints for synthesis.	RTL Design
`rtl/tensor/tensor.sv`	TensorEngine	No	Uses file I/O and non-synth constructs.	High	Rewrite for synthesizable hardware or disable in RTL.	RTL Design
`rtl/cache/l1_cache.sv`	L1_Cache	Yes	Incomplete FSM (reset state not connected).	High	Complete FSM transitions, add reset logic.	RTL Design
`rtl/cache/l2_bank.sv`	L2_Bank	Yes	Correctable; missing `always_ff` sensitivity for SRAM block.	Medium	Fix sensitivity list, instantiate OpenRAM macro.	RTL Design
`rtl/bus/interconnect.sv`	Interconnect	Yes	No issues (standard crossbar logic).	Low	–	RTL Design
`rtl/mem/dram_ctrl.sv`	DRAM_Controller	No	Uses RAM model code (for simulation) instead of port.	High	Replace with actual SRAM macro or blackbox.	RTL/ Memory
`rtl/periph/uart.sv`	UART_Interface	Yes	Good (all synthesizable).	Low	–	RTL Design

File Path	Module	Synthesizable?	Issues	Severity	Required Fixes	Discipline
`specs/clock_tree.sf`	CLK_Tree_Spec	No	Custom `.sf` spec format (non-Verilog). Needs parser.	High	Develop/ obtain translator to SV or re-encode in SV.	CAD Scripting
`tb/sim/top_tb.sv`	Top_SoC_tb	No	Testbench (initial blocks, stimulus, etc.).	Low	Exclude from synthesis; keep for simulation only.	Verificatio

Each module must pass a corresponding acceptance test (simulation-driven functional test) once fixes are applied. High-severity issues (e.g. missing modules, mixed language files) must be resolved before RTL synthesis. These fixes typically involve collaboration between RTL designers and CAD engineers.

2. Target Nodes: Constraints and IP Requirements

We consider implementation at 180 nm, 130 nm (SkyWater), 65 nm and 28 nm. Each node has different timing, power and DFT constraints:

- **Timing/Power:** At 180 nm, clock speeds are modest (~10–50 MHz) and leakage is low. At 130 nm, one can target a few hundred MHz, with moderate leakage. At 65 nm, multi-hundred MHz to ~600 MHz is feasible but leakage grows; at 28 nm, frequencies >1 GHz are possible but static power is significant. For example, 28 nm libraries can give ≈25% higher frequency at ~40% lower dynamic power vs 65 nm【53†L123-L126】. This implies aggressive power management (clock-gating, power domains) will be needed on 28 nm.

- **DFT:** All nodes require design-for-test features. At minimum, plan for scan-chain insertion and JTAG for logic, and built-in self-test (BIST) for memories. Modern flows automatically insert test logic (the LSI 28 nm platform notes “hierarchical design and test insertion capabilities” as standard【74†L1-L4】). Memory macros may include self-test or parity/ECC. The RTL should include scan-friendly flop libraries if possible.

- **Standard Cells & PDK:**

- *130 nm:* The SkyWater SKY130 open PDK provides an open standard-cell library (SKY130_FD_SC_HD/LP) and I/O cells. It is freely accessible and has been used in research and prototypes【34†L346-L352】, but is still in alpha test – suitable for test chips but not yet guaranteed for production. SkyWater’s docs explicitly caution that the open PDK is “a work in progress” (we cite this warning)【34†L346-L352】.
- *180 nm:* No widely-used open PDK exists. Available processes include TSMC/UMC 0.18 µm, or older nodes like UMC180; all require an NDA and PDK license. As a stopgap, open flows sometimes use the SCN4M Submicron (0.18 µm SOI) predictive model or custom “freepdk45” (45 nm) as surrogates. A 180 nm implementation would need licensed standard cells (e.g. 8–10 track libraries) and may reuse SkyWater’s open libraries (they target 130 nm but some scaling info exists).

- *65 nm:* Only commercial nodes (e.g. TSMC 65 nm, GF 65 nm) are available, requiring PDK/IP license. Some universities get TSMC65 through MPW programs, but not open to SMEs freely.
- *28 nm:* Advanced nodes (TSMC 28HPM, GF28SLP) also require expensive licensing (and advanced tool support). I/O libraries for 28 nm (e.g. 1.0 V–3.3 V pads, high-speed SERDES, USB/PCI PHYs) are proprietary.

- **SRAM Compiler:** For prototyping, open-source SRAM compilers like OpenRAM can generate memories in, e.g., 45 nm (freepdk45) or the SCN4M (0.18 µm) tech【28†L249-L252】. The report suggests using OpenRAM to create representative blocks (e.g. 512×32 or 4k×32) for benchmarking. However, actual production silicon at 130 nm/65 nm/28 nm would use foundry-provided memory compilers or pre-characterized macros. SRAM macros (e.g. 256 KB single-port) must be licensed or custom-generated via PDK tools.

- **PLLs and I/O IP:** Phase-locked loops for clock generation are almost never open-source. We must acquire a PLL IP block (often provided with the PDK at extra cost). Similarly, I/O pad libraries (PAD ring cells, ESD protection diodes, analog support cells) come from the foundry. The open SkyWater PDK includes basic I/O cells, but a 28 nm design would need high-speed SERDES or analog PHY IP from a vendor.

- **Licensing Considerations:** In summary, only the 130 nm path has a fully open PDK (Sky130)【34†L346-L352】. For 180/65/28 nm, one must factor in licensing: PDK license fees (often in the 10–100 k€ range for education), NDA agreements, mask costs etc. Standard-cell and memory libraries, as well as tools like Calibre, require separate licenses. Europractice/IC-Link programs can provide MPW access to older nodes (180, 130) with academic pricing, but commercial use may still require industry rates.

3. Yosys + OpenROAD Flow (Experimental)

We propose using the open-source *OpenROAD-flow-scripts* as the baseline flow, with Yosys for logic synthesis and the OpenROAD toolchain for physical design【44†L420-L445】. In a testbed:

- **Tools & Versions:** Use Yosys (e.g. oss-cad-suite release 2024-01-17【3†L259-L264】) and OpenROAD (e.g. v2.0, as in a 2024 release【3†L269-L274】). Install via Docker or apt as in KAMATHAM19's workshop notes【3†L259-L264】【3†L269-L274】. The SkyWater130 open PDK and associated LEF/Lib files (e.g. `sky130_fd_sc_hd`) should be configured in OpenROAD-flow-scripts.

- **Flow Steps:** The pipeline is: read RTL → *Synthesis* (Yosys) → *Floorplan* (define core area, place macros and I/O) → *Placement* (global then detailed) → *Clock-Tree Synthesis* (TritonCTS) → *Routing* (global via FastRoute, detailed via TritonRoute) → *Extraction & STA* (OpenRCX and OpenSTA) → *Signoff checks* (DRC/LVS) → *GDSII*. This follows the standard RTL-GDS flow【44†L420-L445】. For example, in OpenROAD-flow-scripts one writes a config `.tcl` specifying top modules, clock constraints, I/O placement constraints, then invokes the flow script which handles Yosys, TritonCTS, TritonRoute and final GDSII generation automatically【44†L420-L445】.

- **Script Example:** A typical invocation (from KAMATHAM19) is:

```
# Synthesis (Yosys)
yosys -p "read_verilog rtl/top.sv rtl/**/*.sv; synth -top
Top_SilicaFlux; write_verilog out/netlist.v"
# Floorplan & PnR (OpenROAD-flow-scripts)
cd openroad-flow-scripts
make CONFIG=../silicaflux_config.json run
```

(Using an OpenROAD-flow-scripts config JSON to set floorplan/utilization etc.)

- **Substituting Commercial Tools:** For a production flow, several steps would switch to proprietary tools: STA (e.g. Cadence Tempus or Synopsys PrimeTime instead of OpenSTA), DRC/LVS (e.g. Mentor Calibre instead of Magic/KLayout), and PEX (e.g. Calibre xRC). The experimental flow (OpenROAD + KLayout) suffices for preliminary signoff and design closure, but final tapeout must use certified signoff flows.

【62†embed_image】 *Figure: Typical RTL-to-GDSII flow stages (synthesis, floorplan, placement, CTS, routing, extraction and verification)【44†L420-L445】.*

4. SRAM Macro Integration

SilicaFlux requires on-chip SRAM (e.g. L1/L2 caches and scratchpads). We plan the following strategy:

- **Sizes:** Choose representative sizes that cover the range of needs. For instance, one might use 16 KB (512×32) for small scratchpads, 64 KB for L1 caches, and 512 KB or 1 Mb for L2 banks. Exact sizes depend on the architecture; we assume powers-of-two dimensions.

- **OpenRAM:** In SkyWater 130 nm, we can try using OpenRAM【27†L20-L24】 to generate one- or two-port SRAM macros. OpenRAM supports technologies like freepdk45 (45 nm) by default 【28†L249-L252】; community ports exist for skywater130. We would run OpenRAM in verification mode to generate e.g. a 32k×32 (128 KB) single-port SRAM, obtaining a `.lef` and `.lib` to incorporate into the flow. (For example, OpenRAM's docs suggest using commands like `make example_config_scn4m_subm` 【28†L249-L252】, which can be adapted for skywater130 tech.)

- **Area/Latency:** As a rule of thumb, SRAM bitcell area roughly scales with node, and access time improves at smaller nodes. A 512×32 block in freepdk45 might be ~0.005 mm^2 (order-of-magnitude). At 130 nm, bitcell area is ~3–4× larger, so we'd estimate ~0.015 mm^2 (plus periphery) for that block. We will validate these with the OpenRAM-generated PDK models. In 28 nm, the same memory could be ~10× smaller, but again we lack an open 28 nm PDK, so we focus on the 130 nm case.

- **Integration:** Place SRAM macros as "hard blocks" in floorplan. We will allocate regions (as blockages) in the core. The macros' power pins tie into the macro PDN rings【30†L19-L22】. For the OpenROAD flow, we can specify fixed placements of SRAM LEF blocks before standard-cell placement. We also plan to use power and ground straps through these macros (see next section). The memory compiler output (LEF/DEF/SPEF) will be merged into the overall chip GDS.

- **Memory Compiler Experiments:** Besides OpenRAM, one could experiment with alternative open macros (e.g. from efabless caravel repository) or parametric freepdk45 bits. If the L2 cache

is multiported, we may replicate single-port macros or consider two-port OpenRAM (which exists with performance tradeoffs). Timing characterization (read/write cycle time) will be done with the generated `.lib` and STA.

5. Floorplanning, PDN, Placement, CTS and Signoff

Physical design steps and criteria include:

- **Floorplanning:** Define the chip/core outline and aspect ratio. Reserve an IO pad ring if pads are modeled; otherwise, assume an abstract I/O ring. Partition the core into regions: CPU cluster, vector/tensor cluster, memory cluster, etc. Perform "macro floorplan": place large blocks (e.g. SRAM macros, CPU core, coherency engines) first, near related I/Os or each other. Use hierarchical planning for each macro. Then fill the remaining space with standard cells. Insert tapcells at the core edges. For multiclock design (SilicaFlux has e.g. CPU and vector clocks), plan separate clock domains and island cuts if needed.

- **Power Delivery Network (PDN):** Create horizontal and vertical VDD/GND stripes across the core. In the SkyWater flow example, the core used M4/M5 for power straps and M1 for cell rails【30†L19-L22】. We will similarly route thick M4/M5 stripes, connecting to the macro power rings. Ensure each macro has defined power pads ("macro pads") to tie into the grid【30†L19-L22】. Add decoupling capacitor cells at regular intervals (modeled in PDK). Check IR drop limits; if necessary, add additional wells or flip-chip assumptions.

- **Placement:** Run global placement (e.g. OpenROAD's global placer) with placed macros fixed. Then an optimized IO placement step (if not pre-placed). Follow with detailed placement to legalize cell alignments. We expect iterations to meet utilization (target ~50–60% for routing margin) and timing. The placement stage often fixes long nets and fanout by resizing/inserting buffers.

- **Clock Tree Synthesis:** Use TritonCTS (integrated in OpenROAD) to build balanced clock trees. Input is the clock net (from the top module constraints) and the placed flip-flops. TritonCTS buffers and upsizes paths to minimize skew and meet slew requirements【44†L420-L445】. We may insert additional buffers manually in critical paths if needed. Low-skew routing layers are chosen per PDK (e.g. M2 for Skywater).

- **Routing:** Perform global routing (rough channel planning) then detailed routing (actual wire assignment). OpenROAD's FastRoute handles global routing, and TritonRoute does detailed (cutting vias, following preferred directions). Routing automatically fixes antenna diodes on nets as needed (OpenROAD handles "fake antenna" as part of the flow【7†L354-L363】). After routing, merge the final routed DEF into GDSII using Magic or the OpenROAD tools.

- **Parasitic Extraction & STA:** Run OpenROAD's RC extraction (OpenRCX) to produce SPEF. Then run OpenSTA with the extracted netlist and timing libraries to verify timing. Set conservative constraints (e.g. worst-case PVT). Confirm that setup/hold are met with margin. This step is iterative: if timing fails, go back to sizing/buffers.

- **Signoff Checks:** Finally, perform DRC (design-rule check) and LVS (layout-vs-schematic). For an open flow, use Magic or KLayout with a SkyWater rule deck (the SkyWater docs provide Magic/DGRC decks【34†L326-L334】). LVS is done with netgen comparing the extracted netlist to the RTL netlist. Issues found (via a "clean DRC netlist") must be resolved. In our pipeline example,

this follows the pattern of Magic/Netgen usage【7†L372-L380】. We ensure no antenna or via DRC errors remain (the flow automatically inserted fixes).

In summary, the OpenROAD flow covers all major steps【44†L420-L445】. A successful flow yields final GDSII plus signoff reports. Importantly, each stage's results (reports, logs, intermediate DEF/GDS) should be captured for verification.

6. CI Build Scripts and Regression

We will implement a Continuous Integration (CI) pipeline to automate checks. Key elements:

- **Scripts:** Provide a Makefile or shell scripts that invoke Yosys and OpenROAD for the design. For example, `make syn` runs Yosys synthesis and reports area/timing; `make pnr` runs placement and routing. A master script orchestrates the entire OpenROAD flow.

- **CI Gates:** Use GitHub Actions or Jenkins to run these scripts on each commit. Gates include:

- *Linting:* Check HDL syntax (Verilator lint) and style.
- *Synthesis:* Run Yosys; fail if errors or if area/performance regress beyond threshold.
- *Simulation:* Run a suite of testbenches (via Verilator or Questa) to ensure functionality. E.g. a simple program runs on the CPU core (we assume a RISC ISA); vector ops verified; memory read/write checked.

- *Physical:* Optionally run a quick placement (low effort) and static timing analysis to catch major violations early.

- **Test Vectors/Regression:** Develop a regression bench covering all interfaces: CPU instruction tests, vector ops, cache coherency scenarios, interrupts, peripheral I/O. For each module, write unit tests or directed tests. Include assertion checks in simulation where possible. For memory, use model BIST or read-after-write tests. Expected outputs should be golden models or reference (maybe software emulator) for comparison.

- **Artifacts:** Each stage should produce reports: Yosys area/timing report, STA report (slack histogram), DRC/LVS logs. Save these as CI artifacts. Also tag "synthesis-ready RTL" (if passed). If any check fails, the CI pipeline blocks further steps.

This CI setup ensures that the code remains synthesisable and correct through development. It also provides documentation of each stage for review.

7. Proprietary Gaps and Required Interactions

Several critical items are unavailable in a fully open toolchain:

- **PDK Access:** Only 130 nm (Skywater) is open. 65 nm and 28 nm PDKs must be licensed from foundries (typically via NDAs). For example, running in TSMC65 or GF28 requires academic/freeplay programs or corporate budgets. IMEC's IC-Link and Europractice can broker MPW submissions, but still require eligibility (usually academic or EU SME) and PDK agreements.

- **Standard-Cell Libraries:** Sky130's free lib exists. For 65/28 nm, standard-cell libraries must be obtained (e.g. Synopsys DesignWare or foundry-gen). No open 65/28 library is publicly available. We will need to license or adopt generic "virtual" libraries for estimates.

- **SRAM Macros:** While OpenRAM can generate generic SRAMs, real production memories require commercial compilers from ARM, Synopsys, etc. The open PDK's SRAM (if any) is only for demonstration. We anticipate engaging with a memory IP vendor or using the foundry's memory compiler for final tapeout.

- **Analog/IO IP:** PLLs, ADC/DACs, ESD diodes, IO cells (especially high-speed SERDES or PHY IP) must come from the foundry or third-party IP firms. These will likely incur separate license fees.

- **Signoff Tools:** Our flow uses KLayout/Magic for physical checks. For true tapeout signoff, we would require licensed tools (Calibre for DRC/LVS/PEX, Synopsys IC Validator, etc.) and certified rule decks. This gap must be bridged, likely by a foundry partnership or commercial CAD suite.

- **PDN and Analog Verification:** Advanced nodes have complex PDN analysis (IR-drop, electromigration) that is beyond OpenROAD. If we target 28 nm, we'd need SiliconPhysical's Voltus or Mentor's HyperLynx. We note these as future work.

In summary, to fully tape out SilicaFlux we need vendor interactions for PDKs (65/28 nm), IP (PLL, I/O, memories), and signoff tools. For initial 130 nm prototyping, we can proceed with open tools/IP (Sky130 libraries, OpenRAM, etc.) to the extent possible.

8. Module-by-Module Tapeout-Readiness Matrix

Table 1 above lists every RTL file (and key specification files) and assesses readiness. It classifies synthesizable modules (`Yes/No`), notes issues found, severity (High/Medium/Low), fixes needed, responsible discipline (RTL design, CAD, verification) and a brief acceptance test. High-severity items (e.g. missing dependencies, non-synth constructs) must be resolved before synthesis. Once fixes are applied, each module will undergo its "acceptance test" (simulation or small FPGA prototype). For example, after fixing the L1 cache state machine, we will test it via a memory model. The severity column helps prioritize which modules block progress (High = no tapeout until fixed).

9. Work Programme and Milestones

A phased plan with estimated effort and rough costs is proposed:

1. **RTL Cleanup and Simulation (4 PM):** Fix all high/medium issues from Table 1; complete any missing modules. Update or remove `.sf` files by writing converters. Verify all modules via simulation. *Deliverable:* Synthesis-ready RTL netlist (benchmarked in Yosys), self-tests passed.

2. **Prototype ASIC Definition (3 PM):** Define a smaller "SilicaFlux-V" chip (see earlier SilicaFlux-V design) including the minimal CPU core and one vector unit, fewer caches. This clarifies block counts and sizes. *Deliverable:* Block diagram, interfaces, reference RTL for proto-design.

3. **OpenROAD Flow Setup (4 PM):** Configure OpenROAD-flow-scripts with SkyWater130 PDK. Run RTL-to-GDS flow on the prototype design. Identify and fix P&R issues (timing violations, DRC finds). *Deliverable:* Annotated flow scripts, preliminary GDS, DRC/LVS-clean report.

4. **SRAM and Macro Integration (3 PM):** Generate SRAM macros with OpenRAM, integrate into floorplan, rerun place & route. Adjust power grid for macros. *Deliverable:* Floorplan/layout with one memory block placed, power straps anchored.

5. **DFT and Power Checks (2 PM):** Insert scan chains (Yosys `dfflibmap` or OpenROAD DFT scripts), add power decaps, run IR-drop check (simple SPICE run). Resolve any contention. *Deliverable:* Scan-inserted netlist, PDN report.

6. **CI and Regression (2 PM):** Implement CI pipeline (GitHub Actions) with Yosys, OpenROAD, and simulation tests. Achieve automated nightly builds passing all tests. *Deliverable:* CI configuration files, green build badges.

7. **SkyWater 130 nm Tapeout (6 PM, cost ~€30–50k):** Package everything for a SkyWater MPW run (done quarterly via Google/EFabless) – this includes mask-set costs and wafers. Use foundry's MPW program. *Deliverable:* Submitted GDS, design docs, expected silicon mid-program.

8. **65 nm Evaluation (6 PM, cost ~€50–100k):** In parallel or next phase, prepare for a 65 nm tapeout (e.g. via Europractice). This requires securing a 65nm PDK license, adapting flow parameters, and masks. *Deliverable:* Feasibility study, then actual submission to Europractice MPW if funding allows.

9. **28 nm Planning (8 PM, cost ~$100k+):** Long-term, consider 28 nm for high performance. This requires full industry collaboration (PDK license, IP buys, EDA licenses). Initial cost is high (mask NRE ~€200k/mm^2) so likely only if justified. *Deliverable:* RfQ responses, technology port plan.

Overall, we estimate ~30 person-months to full tapeout readiness. Using open tools and the 130 nm MPW route minimizes cost (approx. tens of k€). A full commercial spin at 28 nm with signoff could reach 1M€+ including NRE. This plan aligns with known open-source tapeouts: the OpenROAD project notes that >600 RTL-to-GDS tapeouts have been done (mostly at nodes down to 12 nm) using similar flows 【44†L361-L364】. Our phased approach builds trust at each stage and delivers intermediate demos (e.g. FPGA emulation of SilicaFlux-V with open tools).

Sources: The above plan and descriptions draw on primary flow documentation (e.g. OpenROAD-flow-scripts pipeline【44†L420-L445】 and SkyWater130 PDK notes【34†L346-L352】), SRAM compiler docs 【27†L20-L24】【28†L249-L252】, and foundry info (e.g. MPW schedules【23†L36-L44】). All cited facts come from these references, except where explicitly formulated for SilicaFlux.
